

PRESIDENCY UNIVERSITY

CSE322: Microprocessors Based System Design Laboratory

Lecture 05

Arithmetic and Logical Programming in 8086

Operations, FLAGS, worked programs and debugging

Afzal Hossan

Lecturer · Department of Computer Science and Engineering

Presidency University

Why Arithmetic and Logic Matter

Arithmetic solves numerical problems

- Total price = unit price + delivery cost
- Counter updates and accumulators
- High-precision values spanning registers
- Quotient and remainder calculations

Logic solves bit-oriented problems

- Control bits of a device register
- Combine independent state bits
- Detect equality using comparison
- Test whether a result is zero

An assembly instruction changes data; the FLAGS register explains what happened.

Learning Outcomes and Scope

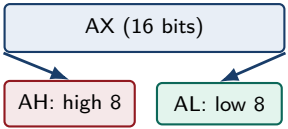
1. Explain operand sizes and 8-/16-bit binary arithmetic.
2. Write correct programs using `ADD`, `ADC`, `SUB`, `SBB`, `INC`, `DEC`.
3. Trace unsigned/signed `MUL` and `DIV` results.
4. Compute bitwise `AND`, `OR`, `XOR`, `NOT` operations.
5. Interpret `CF`, `OF`, `ZF`, `SF`, `PF`, `AF` and use `CMP`.
6. Build, execute, verify and debug assembly experiments.

Boundary with other classes

Shift/rotate instructions and advanced bit masking belong to Class 6; conditional jumps and loops belong to Class 7.

Starting Point: Review from Lecture 4

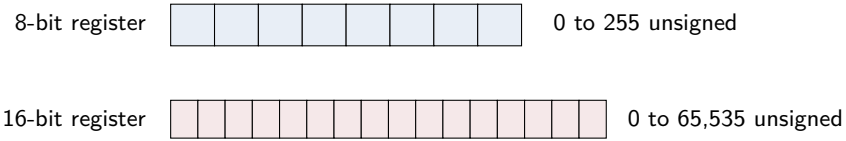
- The destination is generally the *first* operand.
- Registers may be 8-bit (`AL`, `AH`) or 16-bit (`AX`, `BX`).
- An immediate is a literal; `[num]` references memory.
- `MOV` copies data; arithmetic instructions compute.
- The assembler must know each operand's width.




```
MOV AX, 1200h
ADD AL, 03h
```

Result: `AX = 1203h`.

8-bit and 16-bit Arithmetic: Fixed Width



Interpretation matters

Width	Unsigned range	Signed two's-complement range
8-bit	$0 \ldots 255$	$-128 \ldots 127$
16-bit	$0 \ldots 65535$	$-32768 \ldots 32767$

`FFh` means 255 unsigned but -1 signed; the stored bit pattern is identical.

Operand Compatibility: What Is Legal?

Valid 8086 forms

```
ADD AL, BL
ADD AX, 1000h
ADD AX, WORD PTR [num]
SUB BYTE PTR [num], 1
```

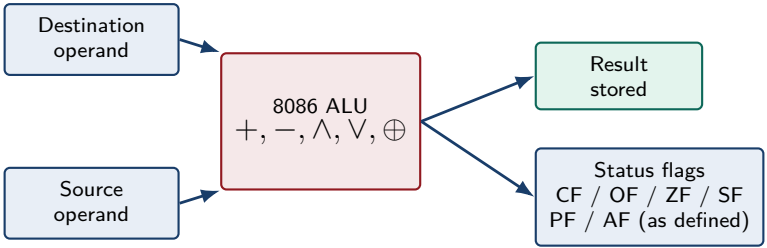
Invalid or ambiguous

`ADD AL, BX`	wrong widths
ADD [a], [b]	two memory operands
MUL 5	immediate not permitted

General rule for two-operand ALU instructions

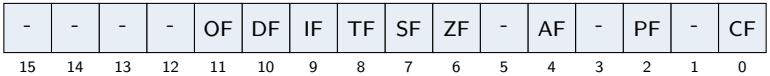
Destination and source are the same width; no general memory-to-memory `ADD/SUB/AND/OR/XOR/CMP`.

A Simplified ALU + FLAGS Dataflow



The same result bits can represent two different numbers: unsigned or signed.

FLAGS Register: Six Arithmetic Status Flags



CF carry / borrow out of width
ZF result is zero
PF parity of low 8 result bits
OF signed overflow
SF sign bit of result
AF carry / borrow from bit 3 to bit 4

8086 nuance

TF, IF and DF are control flags, not status results of normal arithmetic. Undefined flag values must not be used for decisions.

CF vs OF: Same 8 Bits, Different Questions

8-bit ranges: Unsigned: **0 to 255** Signed: -128 **to** $+127$

Example A: Carry without signed overflow

```
FFh + 01h = 00h
   1111 1111
 + 0000 0001
 ------------
   0000 0000
```

Unsigned: $255 + 1 = 256$ (too large).
CF = 1: carry out of bit 7.

Signed: $-1 + 1 = 0$ (fits).
OF = 0: no signed overflow.

Example B: Signed overflow without carry

```
7Fh + 01h = 80h
   0111 1111
 + 0000 0001
 ------------
   1000 0000
```

Unsigned: $127 + 1 = 128$ (fits).
CF = 0: no carry out of bit 7.

Signed: $127 + 1 = 128$ (too large).
OF = 1: 80h represents -128.

Quick rules for 8-bit ADD

CF: Did a carry leave bit 7? **OF:** Did two same-sign inputs produce an opposite-sign result?

ZF, SF, PF and AF with Tiny Examples

Flag	When set	Example (8-bit)
ZF	result = zero	`01h-01h=00h` gives ZF=1
SF	result bit 7 = 1	`00h-01h=FFh` gives SF=1
PF	even number of 1-bits in low byte	`03h` has two 1-bits: PF=1
AF	carry/borrow across bit 3 / 4	`0Fh+01h=10h` gives AF=1

Parity warning

PF does *not* measure parity of the full 16-bit result: only its least significant byte.

Manual Rules: CF, OF and AF (ADD vs SUB)

ADD A, B → R

CF = 1: Unsigned sum exceeds the register width (FF_h for 8 bits; $FFFF_h$ for 16).

OF = 1: A and B have the *same sign*, but R has the *opposite sign*.

AF = 1: Lower-nibble sum $A_{3:0} + B_{3:0} > 15$ (carry from bit 3 to bit 4).

SUB A, B → R

CF = 1: Unsigned $A < B$ (borrow beyond the most significant bit).

OF = 1: A and B have *different signs*, and R has the *opposite sign to A*.

AF = 1: Lower nibble of A is smaller than that of B (borrow from bit 4).

8-bit example	Result	CF	OF	AF
FFh + 01h	00h	1	0	1
7Fh + 01h	80h	0	1	1
00h - 01h	FFh	1	0	1
80h - 01h	7Fh	0	1	1

Exam tip: CF = unsigned carry/borrow; OF = signed overflow; AF = lower-nibble carry/borrow. Rules apply to ADD/SUB without an input carry/borrow.

Worked Flag Trace: Four 8-bit Cases

Operation	Result	CF	OF	ZF	SF	PF	AF
FFh+01h	00h	1	0	1	0	1	1
7Fh+01h	80h	0	1	0	1	0	1
00h−01h	FFh	1	0	0	1	1	1
80h−01h	7Fh	0	1	0	0	0	1

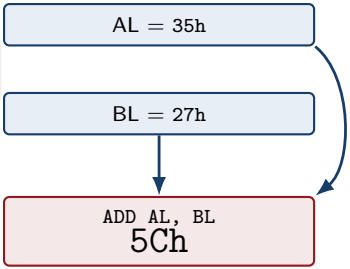
- PF is 1 for 00h (zero 1-bits is even) and FFh (eight 1-bits).
- The same bit pattern can have different signed/unsigned interpretations.
- Use these examples to verify your emulator's FLAGS window.

ADD: Destination ← Destination + Source

Syntax

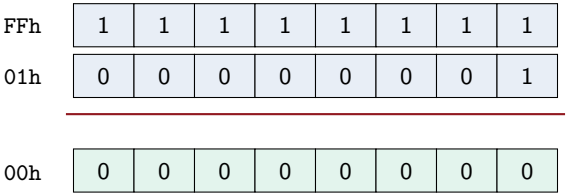
```
ADD destination, source
```

- 8-bit or 16-bit operands.
- Updates CF, OF, SF, ZF, AF, PF.
- Carry beyond bit 7 or 15 goes into CF.



`ADD AL, BL` **changes AL, not BL. Here: CF=0, OF=0.**

ADD in Binary: Follow the Carry



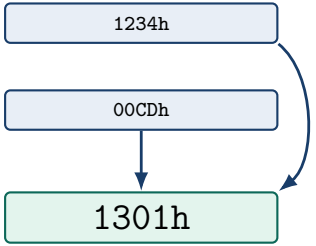
Result

`FFh + 01h = 100h` mathematically; only low 8 bits are stored ($00h$), CF=1.

16-bit ADD: Complete Register Example

```
MOV AX, 1234h
MOV BX, 00CDh
ADD AX, BX        ; AX = 1301h
```

Step	Hex value
Initial AX	1234h
Source BX	00CDh
New AX	1301h



The instruction's width is 16 bits because the destination register is AX.

ADC: Addition with an Incoming Carry

Definition

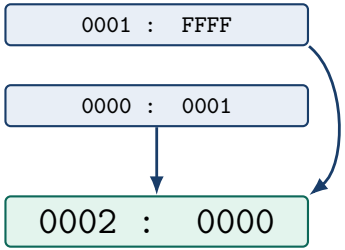
`ADC destination, source`: dest $\leftarrow$ dest + source + $\mathrm{CF}_{\mathrm{in}}$.



Why? Two 16-bit registers can hold a 32-bit value, but a carry from the low word must reach the high word.

Worked 32-bit Addition: ADD then ADC

```
MOV DX, 0001h
MOV AX, 0FFFFh  ; DX:AX = 0001FFFFh
MOV CX, 0000h
MOV BX, 0001h   ; CX:BX = 00000001h
ADD AX, BX      ; AX=0000h, CF=1
ADC DX, CX      ; DX=0002h
```



Crucial ordering

Do not insert any flag-changing instruction between ADD and ADC.

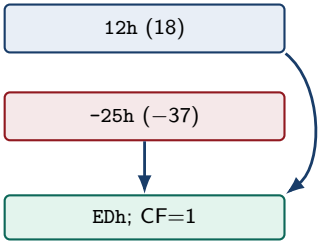
SUB: Subtract and Detect Borrow

Definition

`SUB destination, source`: dest $\leftarrow$ dest $-$ source. CF=1 if *unsigned borrow* is needed.

```
MOV AL, 12h
SUB AL, 25h      ; AL = EDh
```

$12h = 18$, $25h = 37$.
$18 - 37 = -19 \equiv 237 = EDh \pmod{256}$.



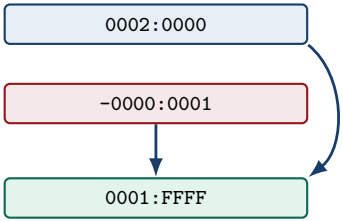
SUB sets CF for a borrow; the result is still stored modulo 2^n.

SBB: Subtract with Borrow

Definition

`SBB destination, source`: dest $\leftarrow$ dest $-$ source $-$ $\mathrm{CF}_{\mathrm{in}}$.

```
MOV DX, 0002h
MOV AX, 0000h
MOV CX, 0000h
MOV BX, 0001h
SUB AX, BX      ; AX=FFFFh, CF=1
SBB DX, CX      ; DX=0001h
```



Practical rule

Use SUB on the low part, SBB on the next part, and continue for larger values.

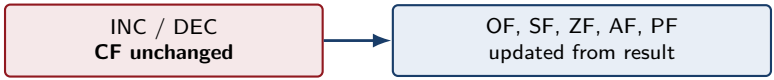
INC and DEC: The Carry Flag Exception

INC destination

`INC AL` means $AL \leftarrow AL + 1$.
Example: `7Fh` $\rightarrow$ `80h`, OF=1.

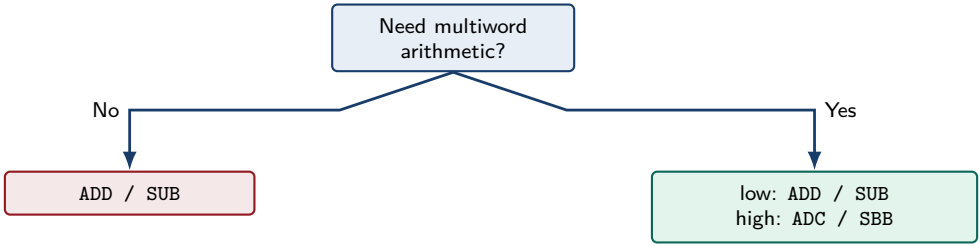
DEC destination

`DEC AL` means $AL \leftarrow AL - 1$.
Example: `80h` $\rightarrow$ `7Fh`, OF=1.



`INC AX` is not identical to `ADD AX,1` if later code depends on CF.

Instruction Choice: ADD or ADC? SUB or SBB?



- Use `INC / DEC` for simple ± 1 adjustments when preserving CF is useful.
- With `ADC/SBB`, always know the incoming CF value.
- `MOV` does not clear CF; `CLC` explicitly clears CF if needed.

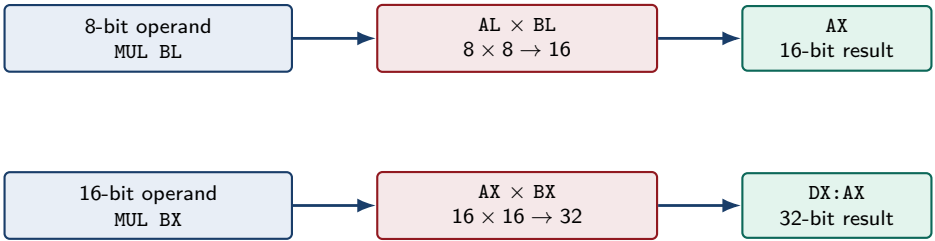
Why Signed Overflow Is Not Carry

Operands	Signed reading	Stored result	CF	OF
`7Fh + 01h`	$127 + 1$	`80h` (-128)	0	1
`FFh + 01h`	$-1 + 1$	`00h` (0)	1	0
`80h - 01h`	$-128 - 1$	`7Fh` (127)	0	1
`00h - 01h`	$0 - 1$	`FFh` (-1)	1	0

Useful reasoning shortcuts

Signed ADD overflow: same-sign inputs give opposite-sign result.
Signed SUB overflow: opposite-sign inputs give result with sign different from destination.

MUL: Implicit Accumulator Operands



Unsigned MUL

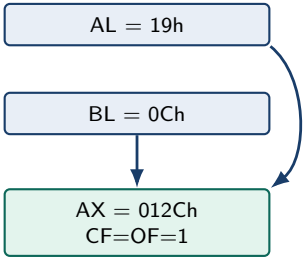
Only one explicit operand. CF=OF=1 if the upper half is nonzero; otherwise both are zero. Other arithmetic status flags are undefined.

8-bit MUL: Result Goes into AX

```
MOV AL, 19h       ; 25 decimal
MOV BL, 0Ch       ; 12 decimal
MUL BL            ; AX = 012Ch
```

$25 \times 12 = 300 = 012Ch$.

AH=01h, AL=2Ch.



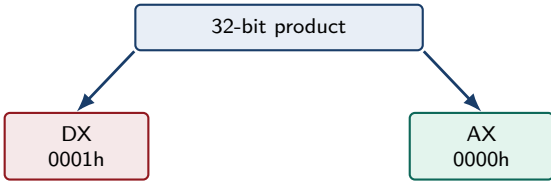
For 8-bit MUL, a product above 255 fits in AX but makes AH nonzero.

16-bit MUL: DX:AX Holds the Product

```
MOV AX, 1000h
MOV BX, 0010h
MUL BX            ; DX:AX = 0001:0000
```

$1000h \times 10h = 10000h$.

Low word goes to AX; high word goes to DX.



Don't discard DX

Reading AX alone would incorrectly report zero for this product!

Signed Multiplication: IMUL (Useful Contrast)

Why a second mnemonic?

Unsigned `MUL` treats all bits as a positive number. 8086 one-operand `IMUL` interprets two's-complement signed values.

```
MOV AL, -5
MOV BL, 6
IMUL BL             ; AX = FFE2h
```

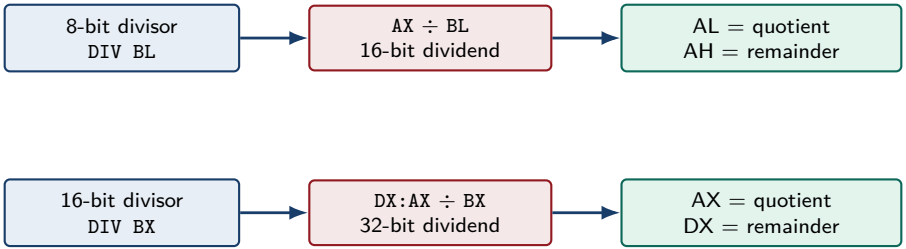
$-5 \times 6 = -30 = FFE2h$.

Mnemonic	Input interpretation	Output
MUL	unsigned	AX or DX:AX
IMUL	signed	AX or DX:AX

On the 8086, `IMUL` has one explicit operand. Later x86 processors add other forms.

Choose signed or unsigned multiplication before you write the instruction.

DIV: Operand Size Determines Where to Look



Unsigned DIV

The dividend is implicit; CF/OF/ZF/SF/AF/PF become undefined.

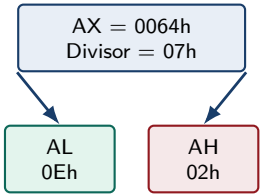
8-bit DIV: Quotient and Remainder

```
MOV AX, 0064h     ; 100 decimal
MOV BL, 07h       ; 7 decimal
DIV BL            ; AL=0Eh, AH=02h
```

$100 = 14 \cdot 7 + 2$

Quotient 14 in AL.

Remainder 2 in AH.

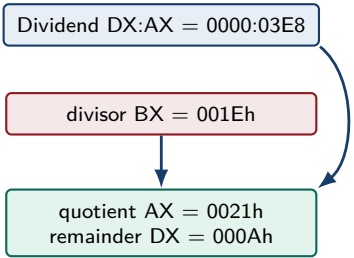


`DIV BL` **does NOT divide only AL. It divides the entire AX by BL.**

16-bit DIV: DX:AX ÷ 16-bit Divisor

```
MOV DX, 0000h
MOV AX, 03E8h    ; 1000 decimal
MOV BX, 001Eh    ; 30 decimal
DIV BX           ; AX=0021h, DX=000Ah
```

$1000 = 33 \cdot 30 + 10.$



Common bug

Clear DX before `DIV BX` *only if* the intended unsigned dividend fits in AX. Never discard a legitimate high word.

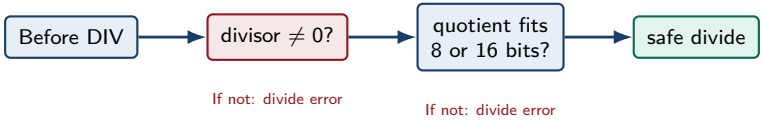
Division Traps: Zero and Quotient Overflow

Divisor is zero

`DIV BL` with BL=0 causes a divide error (interrupt 0); there is no ordinary numeric result.

Quotient does not fit

`AX=0100h; BL=01h; DIV BL`: quotient 256 cannot fit in AL, so divide error occurs.



Signed Division: IDIV, CBW and CWD

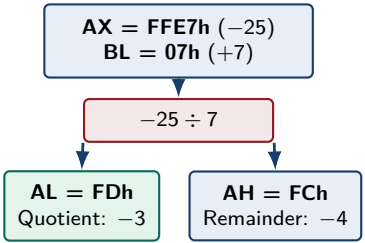
Which registers does IDIV use?

Divisor	Dividend	Quotient	Remainder
IDIV r/m8	signed AX	AL	AH
IDIV r/m16	signed DX:AX	AX	DX

Example: Divide -25 by $+7$

```
MOV AL, -25   ; AL = E7h
CBW           ; AX = FFE7h
MOV BL, 7     ; BL = 07h
IDIV BL       ; signed AX / BL
```

Why CBW? It copies the sign bit of AL into AH, so AX = FFE7h still means -25.



$-25 = (-3) \times 7 + (-4)$. **Quotient truncates toward zero; remainder follows the dividend's sign.**

CWD sign-extends AX into DX:AX for 16-bit divisors.

Multiply vs Divide: Quick Reference

Instruction	Implicit input	Explicit source	Result
`MUL r/m8`	AL	8-bit	AX
`MUL r/m16`	AX	16-bit	DX:AX
`DIV r/m8`	AX	8-bit divisor	AL quotient, AH rem.
`DIV r/m16`	DX:AX	16-bit divisor	AX quotient, DX rem.

Signed equivalents

`IMUL` and `IDIV` use signed inputs; the result register layout is the same as the corresponding unsigned instruction.

Four Bitwise Operations: Truth Table

A	B	A AND B	A OR B	A XOR B
0	0	0	0	0
0	1	0	1	1
1	0	0	1	1
1	1	1	1	0

AND / OR / XOR

Two same-width operands; result replaces destination.

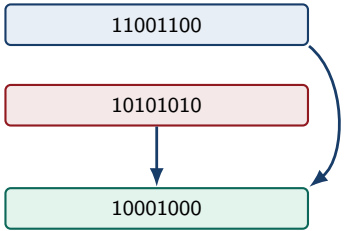
NOT

One operand; every bit is inverted: $\mathrm{NOT}(0) = 1$, $\mathrm{NOT}(1) = 0$.

AND: 1 Only When Both Bits Are 1

```
MOV AL, 0CCh    ; 11001100b
MOV BL, 0AAh    ; 10101010b
AND AL, BL      ; 10001000b = 88h
```

- Result bit $= A_i \wedge B_i$.
- CF=0 and OF=0; ZF/SF/PF are updated.
- AF undefined.

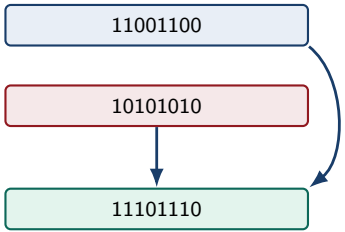


AND is useful when only selected bits should remain visible.

OR: 1 When Either Bit Is 1

```
MOV AL, 0CCh    ; 11001100b
MOV BL, 0AAh    ; 10101010b
OR AL, BL       ; 11101110b = EEh
```

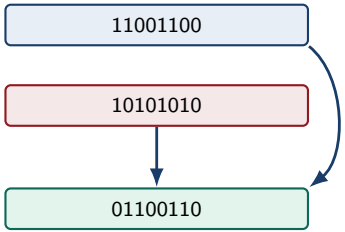
- Result bit $= A_i \vee B_i$.
- Commonly combines existing bits with new ones.
- CF=OF=0; ZF/SF/PF updated; AF undefined.



XOR: 1 When Bits Differ

```
MOV AL, 0CCh    ; 11001100b
MOV BL, 0AAh    ; 10101010b
XOR AL, BL      ; 01100110b = 66h
```

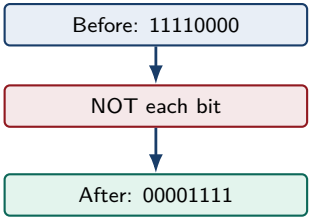
- Result bit = $A_i \oplus B_i$.
- `XOR AX, AX` zeros AX (also changes flags).
- CF=OF=0; ZF/SF/PF updated; AF undefined.



NOT: Complement Every Bit, Keep FLAGS

```
MOV AL, 0F0h     ; 11110000b
NOT AL           ; 00001111b = 0Fh
```

- Unary operation: `NOT destination`.
- Exactly flips all bits in the operand.
- **NOT does not change any FLAGS.**



For an n-bit value, bitwise NOT is $2^n - 1 - x$ (unsigned interpretation).

Bitwise Mini-Lab: Predict Before Running

Instruction	Starting values	Expected destination
`AND AL, BL`	AL=CCh, BL=AAh	88h
`OR AL, BL`	AL=CCh, BL=AAh	EEh
`XOR AL, BL`	AL=CCh, BL=AAh	66h
`NOT AL`	AL=CCh	33h

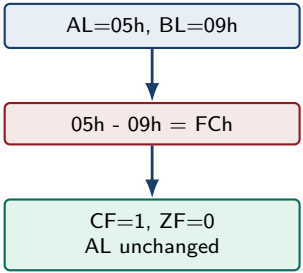
Discussion

If the initial binary pattern is 11001100, explain why these four outputs differ. Which operation leaves the status flags unchanged?

CMP: Subtract Only to Update Flags

```
MOV AL, 05h
MOV BL, 09h
CMP AL, BL      ; flags for 05h - 09h
; AL is still 05h, BL is still 09h
```

- `CMP destination, source` performs an internal subtraction.
- Changes arithmetic status flags just like SUB.
- Does *not* store the subtraction result.



CMP: Equality, Unsigned, Signed

What we want	FLAGS test after `CMP A,B`	Interpretation
Equal?	ZF=1	bit patterns are equal
Unsigned $A < B$	CF=1	unsigned borrow occurred
Unsigned $A > B$	CF=0 and ZF=0	unsigned above
Signed $A < B$	SF $\neq$ OF	signed less-than relation
Signed $A > B$	ZF=0 and SF=OF	signed greater-than relation

Next-class connection

Conditional jump instructions `JE/JB/JL/JA/JG` consume these results; complete branching is taught in Lecture 7.

When the Same CMP Means Opposite Things

```
MOV AL, 0FFh
MOV BL, 01h
CMP AL, BL
```

Stored values are FFh and 01h.

Unsigned

FFh=255, so $255 > 1$.
CF=0 and ZF=0.

Signed

FFh=-1, so $-1 < 1$.
SF $\neq$ OF.

CMP sets flags, not a signed/unsigned mode: the programmer chooses the interpretation.

Instruction-to-FLAGS Cheat Sheet

Instructions	Defined FLAGS changes	Important exceptions
ADD, ADC, SUB, SBB, CMP	CF, OF, SF, ZF, AF, PF	CMP does not store result
INC, DEC	OF, SF, ZF, AF, PF	CF stays unchanged
AND, OR, XOR	CF=0, OF=0; SF, ZF, PF updated	AF undefined
NOT	none	all flags unchanged
MUL, IMUL	CF and OF defined	SF, ZF, AF, PF undefined
DIV, IDIV	none defined	all six status flags undefined

Undefined is not the same as zero or unchanged.

Program 1: Add Two Bytes, Keep 16-bit Sum

```
.MODEL SMALL
.STACK 100h
.DATA
A DB 245
B DB 30
SUM DW ?
.CODE
MAIN PROC
    MOV AX, @DATA
    MOV DS, AX
    MOV AL, A
    MOV AH, 0
    ADD AL, B
    ADC AH, 0
    MOV SUM, AX
    MOV AX, 4C00h
    INT 21h
MAIN ENDP
END MAIN
```

Predict

245+30=275=0113h.

Inspect

SUM = 0113h.
Low byte 13h; high byte 01h.

Stop after `MOV SUM, AX` to inspect memory before DOS termination.

Program 1: A Trace of Its Arithmetic Core

Instruction completed	AL	AH	CF
`MOV AL, A`	F5h	(old)	unchanged
`MOV AH, 0`	F5h	00h	unchanged
`ADD AL, B`	13h	00h	1
`ADC AH, 0`	13h	01h	0
`MOV SUM, AX`	13h	01h	unchanged

Why this works

The byte addition is allowed to overflow; `ADC AH,0` incorporates the outgoing carry into a wider 16-bit result.

Program 2: Subtract and Examine Borrow

```
.MODEL SMALL
.STACK 100h
.DATA
A DB 12h
B DB 25h
DIFF DB ?
.CODE
MAIN PROC
    MOV AX, @DATA
    MOV DS, AX
    MOV AL, A
    SUB AL, B
    MOV DIFF, AL
    ; inspect AL and CF here
    MOV AX, 4C00h
    INT 21h
MAIN ENDP
END MAIN
```

Expected value

12h − 25h = EDh.

Interpret both ways

Unsigned: EDh=237, CF=1.
Signed: EDh=−19, OF=0.

The stored byte alone does not preserve a separate “negative” label.

Program 3: Unsigned 16-bit Product

```
.MODEL SMALL
.STACK 100h
.DATA
A DW 1000h
B DW 0010h
PROD_LO DW ?
PROD_HI DW ?
.CODE
MAIN PROC
    MOV AX, @DATA
    MOV DS, AX
    MOV AX, A
    MUL B              ; DX:AX = A*B
    MOV PROD_LO, AX
    MOV PROD_HI, DX
    MOV AX, 4C00h
    INT 21h
MAIN ENDP
END MAIN
```

Expected product

0001:0000h (32-bit).

Two outputs

PROD_LO=0000h
PROD_HI=0001h

Multiply reads AX implicitly; the 16-bit memory variable B is the explicit factor.

Program 4: Unsigned Division and Remainder

```
.MODEL SMALL
.STACK 100h
.DATA
NUMBER DW 1000
DIVISOR DW 30
QUOT DW ?
REMN DW ?
.CODE
MAIN PROC
    MOV AX, @DATA
    MOV DS, AX
    MOV AX, NUMBER
    XOR DX, DX       ; high word zero
    DIV DIVISOR       ; DX:AX / 30
    MOV QUOT, AX
    MOV REMN, DX
    MOV AX, 4C00h
    INT 21h
MAIN ENDP
END MAIN
```

Check the relation

`1000 = 30 * 33 + 10.`

Expected variables

`QUOT = 0021h (33)`
`REMN = 000Ah (10)`

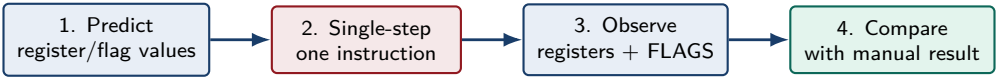
In this example, DX is intentionally cleared because NUMBER is a 16-bit unsigned value.

Program 5: Compute Several Logical Results

```
.MODEL SMALL
.STACK 100h
.DATA
A DB 0CCh
B DB 0AAh
R_AND DB ?
R_OR DB ?
R_XOR DB ?
R_NOT DB ?
.CODE
MAIN PROC
    MOV AX, @DATA
    MOV DS, AX
    MOV AL, A
    AND AL, B
    MOV R_AND, AL
    MOV AL, A
    OR AL, B
    MOV R_OR, AL
    MOV AL, A
    XOR AL, B
    MOV R_XOR, AL
    MOV AL, A
    NOT AL
    MOV R_NOT, AL
    MOV AX, 4C00h
    INT 21h
MAIN ENDP
END MAIN
```

Expected: 88h, EEh, 66h, 33h in the four output variables.

CPU Debugger: Predict, Execute, Verify



- Use an 8086-compatible assembler/simulator (e.g., emu8086 or MASM-compatible environment).
- Watch AL/AH/AX, DX, memory variables, and FLAGS.
- Stop at the instruction immediately after the arithmetic operation.
- Record what changed and explain why; don't just copy the printed result.

Sample Observation Sheet for the Lab

Instruction	Destination after	FLAGS of interest	Interpretation
`ADD AL, BL`	`__`	CF=_ OF=_	carry/overflow?
`ADC AH, 0`	`__`	CF=_	wide sum
`SUB AX, BX`	`__`	CF=_ SF=_	borrow/sign?
`MUL BL`	`AX=__`	CF=_	high byte nonzero?
`DIV BX`	AX=_ DX=_	undefined	quotient/rem.
`CMP AL, BL`	AL unchanged	CF/ZF/SF/OF	compare

Evidence standard

Screenshots should show the actual instruction, registers and FLAGS where applicable.

Fault Diagnosis: Why Programs Give Wrong Results

Symptom	Root cause	Fix
Wrong high word of sum	`ADD` used instead of `ADC`	propagate carry
Wrong 16-bit divide quotient	DX accidentally nonzero	set DX correctly
Missing product upper half	read AX, forgot DX	inspect DX:AX
Unexpected signed result	used `MUL`/`DIV` for negatives	consider IMUL/IDIV
Wrong flags after arithmetic	another instruction changed FLAGS	inspect immediately
Assembler width error	mixed 8-bit and 16-bit operands	match widths

More Debugging: Three Small but Serious Bugs

Bug 1

```
MOV AL,200
ADD AL,100
```

Result wraps to 44. CF=1.

Bug 2

```
MOV AX,1000
DIV BL
```

AX dividend may make quotient overflow AL.

Bug 3

```
CMP AX,BX
ADD CX,1
```

CMP flags have been overwritten.

Single-step through the exact instruction where the first wrong state appears.

Quick Quiz: Predict the Values

1. `AL=7Fh; ADD AL,01h`. What are AL, CF and OF?
2. `AX=FFFFh; ADD AX,1`. What are AX and CF?
3. `AL=05h; SUB AL,09h`. What are AL and CF?
4. `AL=12h; BL=10h; MUL BL`. What is AX?
5. `AX=0064h; BL=07h; DIV BL`. What are AL and AH?
6. `AL=AAh; XOR AL,FFh`. What is AL?

Challenge

For each answer, say which operand changed and which output register(s) must be inspected.

Quick Quiz: Answers and Reasons

No.	Answer	Reason
1	AL=80h, CF=0, OF=1	127+1 overflows signed 8-bit range
2	AX=0000h, CF=1	16-bit carry beyond FFFFh
3	AL=FCh, CF=1	5-9 needs unsigned borrow
4	AX=0120h	$18 \times 16 = 288$ (result uses AX)
5	AL=0Eh, AH=02h	$100 = 14 \times 7 + 2$
6	AL=55h	every bit toggled by XOR FFh

Lab Experiment 5: Required Practical Tasks

1. **A.** Add two 8-bit values and store the full 16-bit sum. Show carry tracing.
2. **B.** Subtract two unsigned 16-bit values and examine CF and the stored result.
3. **C.** Multiply two 16-bit values and record the full DX:AX product.
4. **D.** Divide a 16-bit unsigned number; store quotient and remainder.
5. **E.** Evaluate AND, OR, XOR, NOT on provided byte values.
6. **F.** Compare two bytes using CMP and report unsigned and signed relations.

Expected output

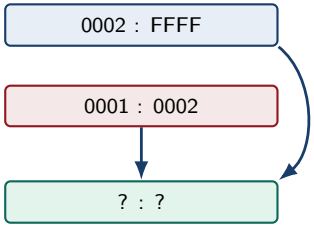
Source code, predicted values, screenshots, FLAGS interpretation and a short error analysis.

Guided Challenge A: Two-Word Calculator

Given two 32-bit unsigned values

$A = 0002\text{FFFFh}$, $B = 00010002\text{h}$.

Store A in DX:AX and B in CX:BX.
Use `ADD` and `ADC` to find their sum.

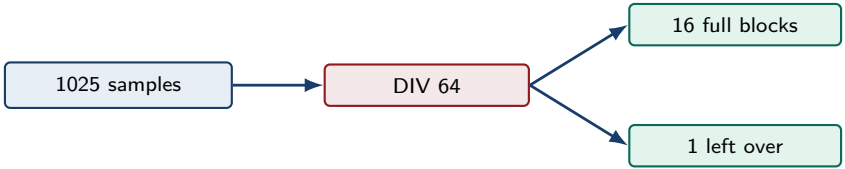


Check: answer should be `0004:0001h`; low-word carry is essential.

Guided Challenge B: Remainder Is Useful

Problem

A sensor transmits 1025 samples. Pack them into full blocks of 64 samples. How many complete blocks and how many leftover samples?



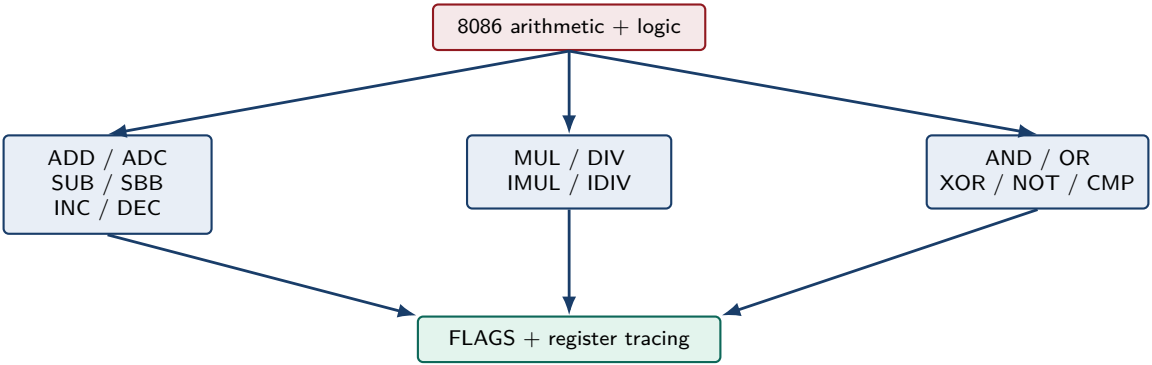
Implement: `MOV DX,0; MOV AX,1025; MOV BX,64; DIV BX`. Inspect AX and DX.

Assessment: What a Strong Lab Report Includes

- Experiment title and objective
- Software / environment
- Theory and relevant syntax
- Algorithm and flowchart
- Complete commented assembly code
- Anticipated outputs and FLAGS
- Debugging and observation screenshots
- Actual output or memory values
- Manual verification by calculation
- Error analysis and corrections
- Conclusion and viva questions
- Correct identification of signed/unsigned operations

Explain *why* each FLAG is set rather than reporting only the final answer.

Concept Map: What You Have Learned



Takeaway

Correct assembly programming requires three checks: *operand width*, *destination location*, and *FLAGS meaning*.

Viva Practice: Explain, Do Not Memorize

1. Why does `ADC` matter when adding numbers wider than a register?
2. What is the difference between CF and OF?
3. Why is `INC` not always equivalent to `ADD dest,1`?
4. Where does an 8-bit `MUL` store its product?
5. Why must we inspect DX before `DIV BX`?
6. Which FLAGS change after `NOT`? After `AND`?
7. What does `CMP` change, and what does it preserve?

Final Recap: Five Rules You Must Remember

1. **Choose the right width**: 8-bit vs 16-bit affects every computation.
2. **Use CF to chain words**: `ADD`/`ADC` and `SUB`/`SBB`.
3. **Know implicit registers**: AX and DX:AX for multiply/divide.
4. **Interpret FLAGS correctly**: CF is unsigned; OF is signed.
5. **Trace the real machine**: register/memory values and flag updates must agree with your calculation.

Next: Lecture 6

Bit manipulation, bit testing, masking in depth, and shift/rotate instructions.



Code. Trace. Verify.

Mastering arithmetic and logic on the 8086

Questions and Lab Discussion

Afzal Hossan · Presidency University · CSE322